```
      )
      .flatMap(orderRepository::save);                                    ❹
  }

  public static Order buildAcceptedOrder(Book book, int quantity) {
    return Order.of(book.isbn(), book.title() + " - " + book.author(),
      book.price(), quantity, OrderStatus.ACCEPTED);                      ❺
  }

  public static Order buildRejectedOrder(String bookIsbn, int quantity) {
    return Order.of(bookIsbn, null, null, quantity, OrderStatus.REJECTED);
  }
}
```

❶ Calls the Catalog Service to check the book's availability

❷ If the book is available, it accepts the order.

❸ If the book is not available, it rejects the order.

❹ Saves the order (either as accepted or rejected)

❺ When an order is accepted, we specify ISBN, book name (title + author), quantity, and status. Spring Data takes care of adding the identifier, version, and audit metadata.

Let's try that out. First, ensure the PostgreSQL container is up and running by executing the following command from the folder where you keep your Docker Compose configuration (polar-deployment/docker):

```
$ docker-compose up -d polar-postgres
```

Then build and run both Catalog Service and Order Service (`./gradlew bootRun`).

WARNING If you're using an Apple Silicon computer, the application logs from Order Service might include some warnings related to DNS resolutions in Netty. In this specific case, the application should still work correctly. If you experience issues, you can add the following additional dependency to the Order Service project as `runtimeOnly` to fix the problem: `io.netty:netty-resolver-dns-native-macos:4.1.79.Final:osx-aarch_64`.

Finally, send an order for one of the books created in Catalog Service at startup time. If the book exists, the order should be accepted: